

Hoisting

What is Hoisting (Real Meaning)?

Most tutorials say:

“Variables are moved to the top.”

That is **WRONG**.

JavaScript does **NOT move code**.

Instead:

Before executing your code, the JS engine scans it and registers declarations in memory.

This phase is called:

🔄 Creation Phase (Memory Allocation Phase)

Memory registration before execution is hoisting

Hoisting is simply what happens in **Phase 1**.

💧 Why JS Does Hoisting?

Because JavaScript uses something called:

Lexical Environment + Execution Context

The engine must know:

- What variables exist
- What functions exist
- Scope relationships

BEFORE running code.

So it scans once, builds memory map, then executes.

🔍 What Gets Hoisted?

Declaration Type	Hoisted?	Initialized?
var	☑ Yes	undefined
let	☑ Yes	✗ No (TDZ)
const	☑ Yes	✗ No (TDZ)
Function Declaration	☑ Yes	Full function
Function Expression	✗ No	Acts like variable

🔥 What is TDZ Really? Temporal Dead Zone

TDZ is NOT a place.

It is a **time window** during execution when the variable exists but cannot be accessed.

Scope starts ——— TDZ ——— Declaration line ——— Normal usage

🚀 Function Hoisting (Different Behavior!)

Function declarations are **fully hoisted with their body**.

```
greet();
```

```
function greet() {
  console.log("Hello");
}
```

Works because during Creation Phase:

```
greet: function() {...}
```

The function is already stored in memory.

✗ Function Expression is NOT Hoisted Like That

```
greet();
```

```
var greet = function () {
  console.log("Hello");
}
```

```
};
```

Creation Phase:

greet: undefined

Execution:

TypeError: greet is not a function

Because it's treated like a var, not like a function.

Mental Model to Remember

Think of JS like this:

Step 1: Scan the code → Register everything

Step 2: Execute the code → Assign & run

NOT:

"Move things to top"

Interview Trap Example

```
var a = 1;
```

```
function test() {  
  console.log(a);  
  var a = 2;  
}
```

```
test();
```

Most expect 1.

Actual output:

undefined

Why?

Inside test, JS hoists:

```
function test() {  
  var a;    // hoisted locally  
  console.log(a);  
  a = 2;  
}
```

Local a shadows global a.

Another Classic Trap

```
let a = 10;  
  
{  
  console.log(a);  
  let a = 20;  
}
```

This throws error.

Because block creates new scope:

Creation Phase inside block:

a: <uninitialized>

So it DOES NOT look at outer a.

You're inside TDZ of inner a.